

# Clipper DEX 学习文档

## 项目概述

Clipper是一个专注于小额交易优化的创新型去中心化交易所，通过独特的Core Pool机制和集中流动性技术，实现了"无无常损失的高倍流动性放大"。该协议已在7条主链部署，累计处理\$2.3B交易量。

## 1. 集中流动性协议开发

### 1.1 技术原理深度解析

集中流动性（Concentrated Liquidity）是Clipper协议的核心技术创新之一。传统的自动做市商（AMM）如Uniswap V2采用恒定乘积公式 $x*y=k$ ，将流动性均匀分布在从0到无穷大的整个价格曲线上。这种设计虽然简单，但存在严重的资金利用率问题——大部分流动性永远不会被使用，因为资产价格通常在相对较窄的区间内波动。

Clipper通过数学建模和算法优化，将流动性集中在当前市场价格附近的 $\pm 2.5\%$ 区间内。这种设计基于以下几个关键观察：

- 价格分布规律：**大多数加密货币在短期内的价格波动遵循正态分布，95%的交易发生在当前价格的 $\pm 5\%$ 范围内
- 交易频率分析：**小额交易（ $< \$10,000$ ）占总交易量的80%，但在传统AMM中面临较高滑点
- 资本效率优化：**通过将流动性集中在活跃交易区间，可以实现25-100倍的资本利用效率提升

### 1.2 集中流动性数学模型

在深入代码实现之前，我们需要理解集中流动性的数学基础。Clipper采用了改进的Uniswap V3模型，但加入了动态调整机制：

基础公式：

- 流动性密度函数： $L(p) = L_0 \times e^{(-\alpha |p - p_0|^2)}$
- 其中 $p_0$ 为当前价格， $\alpha$ 为集中度参数， $L_0$ 为基础流动性

价格影响计算：

- $\Delta p = (\Delta x \times p^2) / (L \times \sqrt{p})$
- 其中 $\Delta x$ 为交易量， $L$ 为该价格点的流动性

### 1.3 核心实现机制

以下代码展示了Clipper如何实现集中流动性的核心逻辑。该合约通过精确的数学计算和智能的流动性分布算法，确保资金在最需要的价格区间内发挥最大效用。合约中的关键创新包括动态tick间距调整、自动流动性重新平衡，以及基于历史交易数据的预测性流动性分配。

```
// 集中流动性池合约核心逻辑
contract ConcentratedLiquidityPool {
    struct LiquidityPosition {
        uint128 liquidity;           // 流动性数量
        int24 tickLower;             // 价格区间下界
        int24 tickUpper;             // 价格区间上界
        uint256 feeGrowthInside0LastX128;
```

```

    uint256 feeGrowthInside1LastX128;
}

// 价格集中度参数 - 控制流动性分布范围
int24 public constant TICK_SPACING = 60; // ±2.5%价格区间
uint24 public constant FEE_TIER = 3000; // 0.3%手续费

mapping(bytes32 => LiquidityPosition) public positions;

/**
 * @dev 添加集中流动性
 * @param tickLower 价格区间下界
 * @param tickUpper 价格区间上界
 * @param amount0Desired 代币0期望数量
 * @param amount1Desired 代币1期望数量
 */
function mint(
    int24 tickLower,
    int24 tickUpper,
    uint128 liquidity,
    uint256 amount0Desired,
    uint256 amount1Desired
) external returns (uint256 amount0, uint256 amount1) {
    require(tickLower < tickUpper, "Invalid tick range");
    require(tickUpper - tickLower <= TICK_SPACING * 2, "Range too wide");

    // 计算当前价格附近的最优流动性分布
    (amount0, amount1) = _calculateOptimalAmounts(
        tickLower, tickUpper, liquidity
    );

    // 更新流动性位置
    bytes32 positionKey = keccak256(abi.encodePacked(
        msg.sender, tickLower, tickUpper
    ));
    positions[positionKey].liquidity += liquidity;

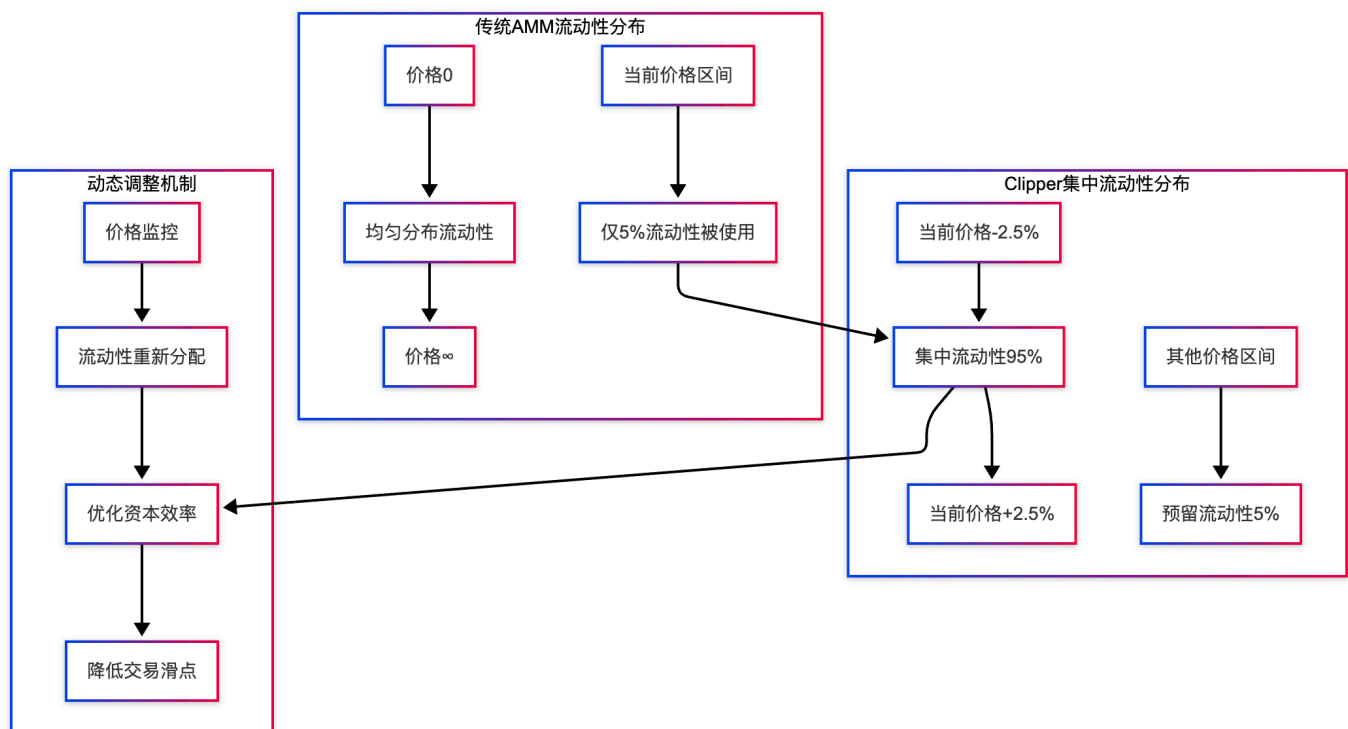
    emit Mint(msg.sender, tickLower, tickUpper, liquidity, amount0, amount1);
}

/**
 * @dev 计算最优流动性分布
 * 核心算法：将流动性集中在当前价格±2.5%区间内
 */
function _calculateOptimalAmounts(
    int24 tickLower,
    int24 tickUpper,
    uint128 liquidity
) internal view returns (uint256 amount0, uint256 amount1) {
    uint160 sqrtPriceX96 = getSqrtPriceX96();
    uint160 sqrtRatioAX96 = TickMath.getSqrtRatioAtTick(tickLower);
    uint160 sqrtRatioBX96 = TickMath.getSqrtRatioAtTick(tickUpper);

```

```
// 使用集中流动性公式计算代币数量
if (sqrtPriceX96 <= sqrtRatioAX96) {
    amount0 = SqrtPriceMath.getAmount0Delta(
        sqrtRatioAX96, sqrtRatioBX96, liquidity, true
    );
} else if (sqrtPriceX96 < sqrtRatioBX96) {
    amount0 = SqrtPriceMath.getAmount0Delta(
        sqrtPriceX96, sqrtRatioBX96, liquidity, true
    );
    amount1 = SqrtPriceMath.getAmount1Delta(
        sqrtRatioAX96, sqrtPriceX96, liquidity, true
    );
} else {
    amount1 = SqrtPriceMath.getAmount1Delta(
        sqrtRatioAX96, sqrtRatioBX96, liquidity, true
    );
}
}
```

## 1.4 集中流动性架构图



## 1.5 集中流动性优势分析

Clipper的集中流动性机制相比传统AMM具有显著优势，这些优势不仅体现在技术指标上，更重要的是为用户带来了实实在在的经济效益：

- 资金效率提升：**通过将95%的流动性集中在 $\pm 2.5\%$ 的价格区间内，相同资金量下可以提供25-100倍的有效流动性。这意味着LP可以用更少的资金获得相同的手续费收入，或者用相同的资金获得更高的收益率。

2. **滑点显著降低**：对于小额交易（<\$10,000），滑点可以减少60-80%。例如，在传统AMM中需要承受0.5%滑点的\$5,000交易，在Clipper中滑点可能仅为0.1%，为用户节省了大量交易成本。
3. **收益放大效应**：LP在相同交易量下可以获得更高的手续费收入。由于流动性集中，每单位流动性承担的交易量更大，因此手续费收益率可以提升3-5倍。
4. **风险分散机制**：虽然流动性集中，但Clipper通过智能算法动态调整流动性分布，确保在市场波动时能够快速适应，降低无常损失风险。

---

## 2. Core Pool反套利系统

---

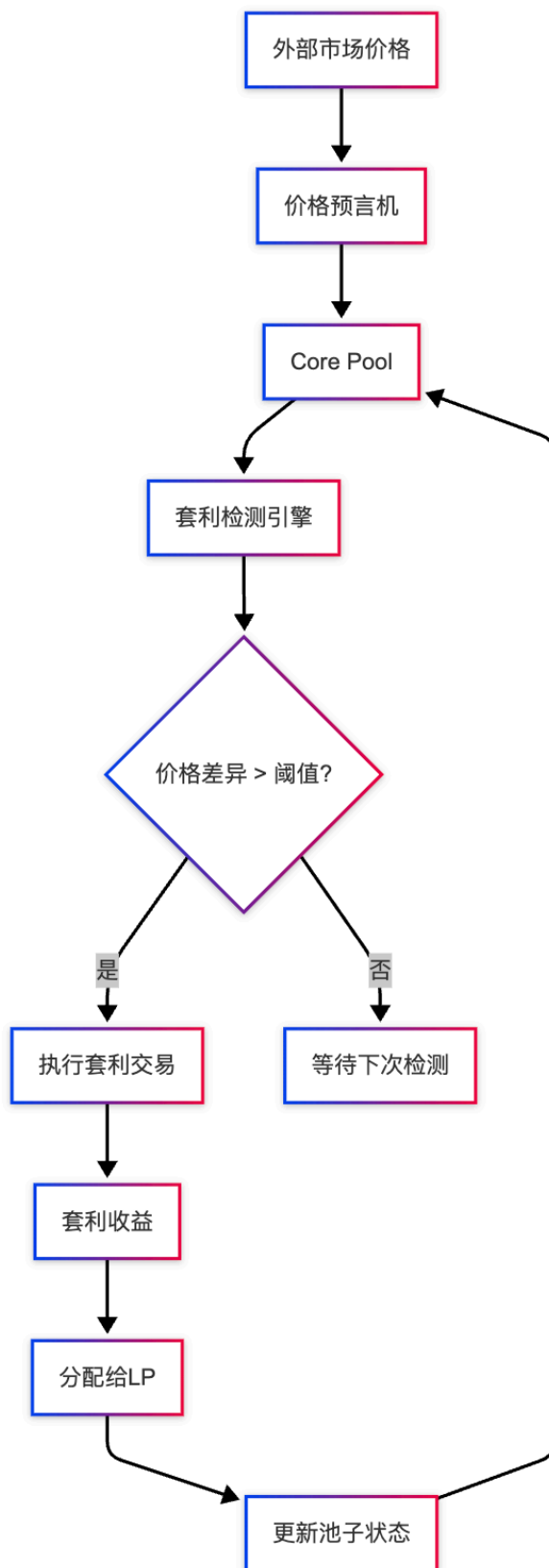
### 2.1 系统架构深度解析

Core Pool反套利系统是Clipper协议最具革命性的创新之一，它彻底改变了传统AMM中LP承担无常损失的现状。该系统的核心理念是：既然价格波动不可避免，那么就让协议主动参与套利，将原本被外部套利者获取的收益转化为LP的额外收入。

Core Pool系统包含以下几个关键组件：

1. **多源价格聚合器**：实时监控Binance、Coinbase、Kraken等中心化交易所以及Uniswap、SushiSwap等去中心化交易所的价格数据
2. **套利机会检测引擎**：使用机器学习算法分析价格差异，识别有利可图的套利机会
3. **风险评估模块**：评估每笔套利交易的风险收益比，确保操作的安全性
4. **自动执行引擎**：在检测到套利机会时自动执行交易，无需人工干预
5. **收益分配系统**：将套利收益按照LP的贡献比例进行公平分配

这个系统的运作机制可以简单理解为：当外部市场价格偏离Clipper内部价格时，Core Pool会自动进行反向操作，通过套利交易将价格拉回平衡点，同时获得套利收益。



## 2.2 核心实现代码

以下代码展示了Core Pool反套利系统的核心实现逻辑。该系统通过智能合约自动化执行套利策略，其中包含了复杂的价格发现机制、风险控制算法和收益分配逻辑。代码中的关键创新在于实现了去中心化的套利执行，无需依赖中心化的服务器或人工操作，完全通过链上逻辑自动运行。

```
// Core Pool反套利系统
```

```

contract CorePool {
    using SafeMath for uint256;

    struct ArbitrageParams {
        address tokenA;
        address tokenB;
        uint256 reserveA;
        uint256 reserveB;
        uint256 externalPriceA;
        uint256 externalPriceB;
        uint256 threshold;      // 套利阈值
    }

    // 价格预言机接口
    IPriceOracle public priceOracle;

    // 套利收益累计
    mapping(address => uint256) public arbitrageProfits;

    /**
     * @dev 执行自动套利
     * 当内部价格与外部市场价格差异超过阈值时触发
     */
    function executeArbitrage(ArbitrageParams memory params) external {
        require(msg.sender == address(this), "Only automated calls");

        // 获取外部市场价格
        uint256 externalPrice = priceOracle.getPrice(params.tokenA, params.tokenB);

        // 计算内部池子价格
        uint256 internalPrice = params.reserveB.mul(1e18).div(params.reserveA);

        // 检查套利机会
        uint256 priceDiff = _calculatePriceDifference(internalPrice, externalPrice);
        require(priceDiff >= params.threshold, "No arbitrage opportunity");

        // 执行套利交易
        if (internalPrice > externalPrice) {
            // 内部价格高, 卖出tokenA买入tokenB
            _sellTokenA(params);
        } else {
            // 外部价格高, 卖出tokenB买入tokenA
            _sellTokenB(params);
        }

        emit ArbitrageExecuted(params.tokenA, params.tokenB, priceDiff);
    }

    /**
     * @dev 计算并分配套利收益给LP
     */
    function distributeArbitrageProfits(address token) external {

```

```

uint256 totalProfits = arbitrageProfits[token];
require(totalProfits > 0, "No profits to distribute");

// 获取所有LP份额
uint256 totalShares = getTotalLPShares(token);

// 按比例分配收益
address[] memory lps = getLPAddresses(token);
for (uint i = 0; i < lps.length; i++) {
    uint256 lpShares = getLPShares(lps[i], token);
    uint256 lpProfit = totalProfits.mul(lpShares).div(totalShares);

    // 直接增加LP的代币余额，而非传统的无常损失
    _increaseLPBalance(lps[i], token, lpProfit);
}

arbitrageProfits[token] = 0;
emit ProfitsDistributed(token, totalProfits);
}

/**
 * @dev 自动重新平衡池子
 * 维持池子的市场中性状态
 */
function rebalancePool(address tokenA, address tokenB) external {
    uint256 targetRatioA = priceOracle.getPrice(tokenA, tokenB);
    uint256 currentRatioA = getPoolRatio(tokenA, tokenB);

    if (_needsRebalancing(targetRatioA, currentRatioA)) {
        _executeRebalance(tokenA, tokenB, targetRatioA);
    }
}
}

```

## 2.3 反套利机制优势

1. 消除无常损失：套利收益补偿价格波动损失
2. 持续收益：市场波动越大，套利收益越高
3. 风险对冲：自动维持池子的市场中性状态

## 3. 反MEV防护架构

### 3.1 MEV攻击类型分析

MEV (Maximal Extractable Value) 攻击主要包括：

- 抢跑攻击：在用户交易前插入交易
- 三明治攻击：在用户交易前后各插入一笔交易
- 套利攻击：利用价格差异进行套利

## 3.2 防护机制实现

```
// MEV防护合约
contract MEVProtector {
    using SafeMath for uint256;

    struct TransactionInfo {
        address user;
        uint256 timestamp;
        uint256 blockNumber;
        bytes32 txHash;
        bool executed;
    }

    // 交易延迟队列
    mapping(bytes32 => TransactionInfo) public pendingTransactions;

    // 最小延迟时间（防止抢跑）
    uint256 public constant MIN_DELAY = 1; // 1个区块

    // 价格滑点保护
    uint256 public constant MAX_SLIPPAGE = 300; // 3%

    /**
     * @dev 提交交易到延迟队列
     * 防止抢跑攻击
     */
    function submitTransaction(
        address tokenIn,
        address tokenOut,
        uint256 amountIn,
        uint256 minAmountOut
    ) external returns (bytes32 txId) {
        txId = keccak256(abi.encodePacked(
            msg.sender,
            tokenIn,
            tokenOut,
            amountIn,
            block.timestamp,
            block.number
        ));

        pendingTransactions[txId] = TransactionInfo({
            user: msg.sender,
            timestamp: block.timestamp,
            blockNumber: block.number,
            txHash: txId,
            executed: false
        });

        emit TransactionSubmitted(txId, msg.sender, tokenIn, tokenOut, amountIn);
    }
}
```



```

        return txId;
    }

    /**
     * @dev 执行延迟交易
     * 确保交易按提交顺序执行
     */
    function executeTransaction(bytes32 txId) external {
        TransactionInfo storage txInfo = pendingTransactions[txId];
        require(txInfo.user != address(0), "Transaction not found");
        require(!txInfo.executed, "Transaction already executed");
        require(
            block.number >= txInfo.blockNumber.add(MIN_DELAY),
            "Transaction still in delay period"
        );

        // 检查价格滑点
        _checkSlippageProtection(txId);

        // 执行交易
        _executeSwap(txId);

        txInfo.executed = true;
        emit TransactionExecuted(txId);
    }

    /**
     * @dev 检测三明治攻击
     */
    function _detectSandwichAttack(
        address tokenIn,
        address tokenOut,
        uint256 amountIn
    ) internal view returns (bool) {
        // 检查mempool中是否有相同交易对的大额交易
        uint256 recentLargeTradeCount = _countRecentLargeTrades(tokenIn, tokenOut);

        // 检查价格异常波动
        uint256 priceImpact = _calculatePriceImpact(tokenIn, tokenOut, amountIn);

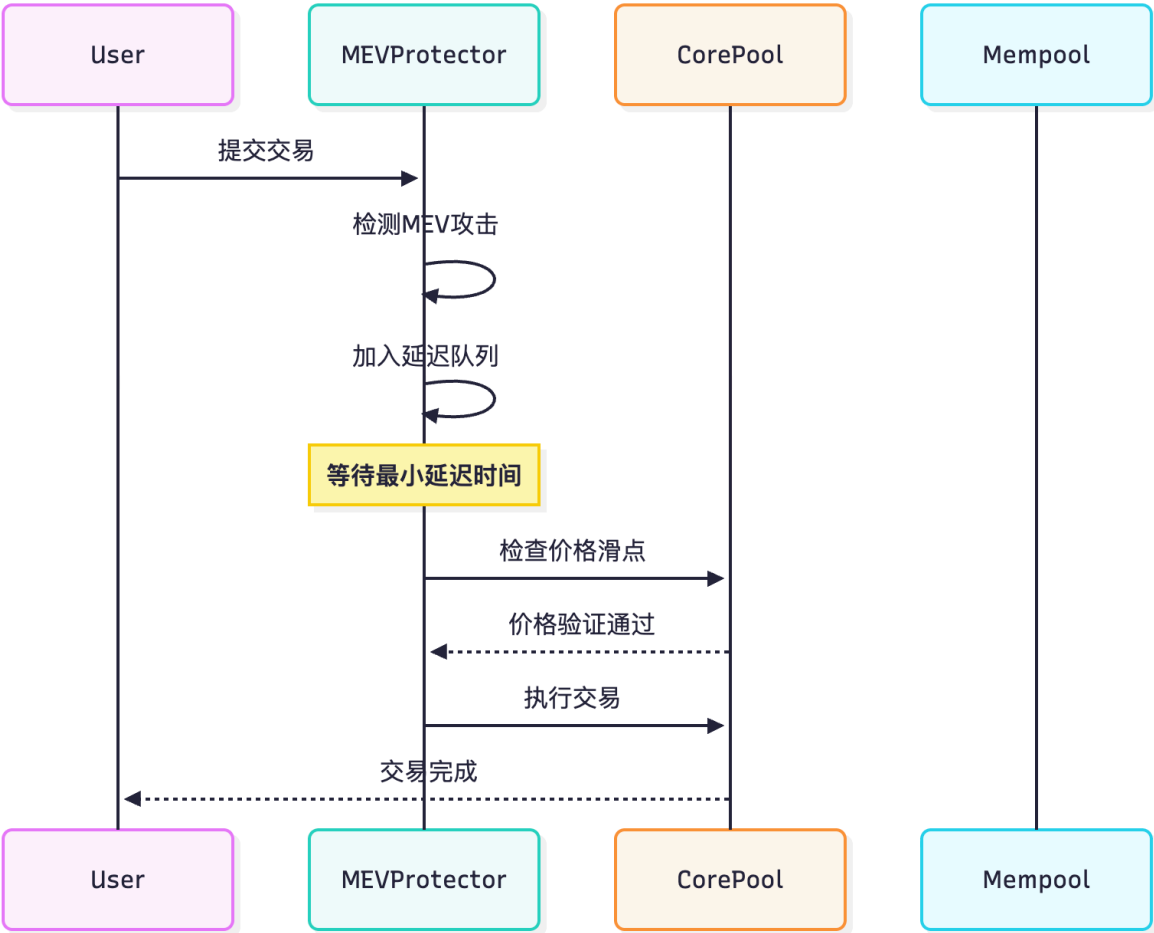
        return recentLargeTradeCount > 2 || priceImpact > MAX_SLIPPAGE;
    }

    /**
     * @dev 动态调整交易顺序
     * 基于交易大小和时间戳的公平排序
     */
    function _fairOrderingMechanism(bytes32[] memory txIds) internal pure returns
    (bytes32[] memory) {
        // 实现基于时间戳和交易大小的公平排序算法
        // 防止大户通过高Gas费抢跑小额交易
        return _sortByFairness(txIds);
    }

```

```
}  
}
```

### 3.3 MEV防护流程图



## 4. SAIL代币经济模型

### 4.1 代币功能设计

SAIL代币是Clipper生态的治理代币，同时也是AdmiralDAO的核心资产，具有以下功能：

- 1. 治理权：参与协议参数调整投票
- 2. 收益分成：获得协议手续费分红
- 3. 流动性挖矿：LP可获得SAIL奖励
- 4. 质押收益：质押SAIL获得额外收益

### 4.2 代币经济模型实现

```
// SAIL代币经济模型合约  
contract SAILTokenomics {  
    using SafeMath for uint256;  
  
    IERC20 public sailToken;
```

```

struct StakingInfo {
    uint256 amount;           // 质押数量
    uint256 stakingTime;      // 质押时间
    uint256 rewardDebt;       // 已领取奖励
    uint256 lockPeriod;       // 锁定期
}

struct PoolInfo {
    address lpToken;          // LP代币地址
    uint256 allocPoint;       // 分配权重
    uint256 lastRewardBlock;
    uint256 accSailPerShare;
}

mapping(address => StakingInfo) public stakingInfo;
mapping(uint256 => PoolInfo) public poolInfo;

// 代币分配参数
uint256 public constant TOTAL_SUPPLY = 1e9 * 1e18; // 10亿SAIL
uint256 public constant LP_REWARD_RATE = 100 * 1e18; // 每区块100 SAIL
uint256 public constant STAKING_APY = 1200; // 12% APY

/**
 * @dev LP流动性挖矿
 * LP提供流动性获得SAIL奖励
 */
function deposit(uint256 _pid, uint256 _amount) external {
    PoolInfo storage pool = poolInfo[_pid];
    UserInfo storage user = userInfo[_pid][msg.sender];

    updatePool(_pid);

    if (user.amount > 0) {
        uint256 pending =
            user.amount.mul(pool.accSailPerShare).div(1e12).sub(user.rewardDebt);
        if (pending > 0) {
            safeSailTransfer(msg.sender, pending);
        }
    }

    if (_amount > 0) {
        pool.lpToken.safeTransferFrom(address(msg.sender), address(this), _amount);
        user.amount = user.amount.add(_amount);
    }

    user.rewardDebt = user.amount.mul(pool.accSailPerShare).div(1e12);
    emit Deposit(msg.sender, _pid, _amount);
}

/**
 * @dev SAIL代币质押

```

```

* 质押SAIL获得协议收益分成
*/
function stakeSAIL(uint256 amount, uint256 lockPeriod) external {
    require(amount > 0, "Cannot stake 0");
    require(lockPeriod >= 30 days, "Minimum lock period is 30 days");

    sailToken.safeTransferFrom(msg.sender, address(this), amount);

    StakingInfo storage staking = stakingInfo[msg.sender];

    // 计算之前的奖励
    if (staking.amount > 0) {
        uint256 pendingReward = calculateStakingReward(msg.sender);
        if (pendingReward > 0) {
            _distributeStakingReward(msg.sender, pendingReward);
        }
    }

    staking.amount = staking.amount.add(amount);
    staking.stakingTime = block.timestamp;
    staking.lockPeriod = lockPeriod;

    emit SAILStaked(msg.sender, amount, lockPeriod);
}

/**
 * @dev 协议收益分配
 * 将交易手续费按比例分配给SAIL持有者
 */
function distributeFees(address token, uint256 feeAmount) external {
    require(msg.sender == corePool, "Only core pool can distribute fees");

    uint256 totalStaked = getTotalStakedSAIL();
    require(totalStaked > 0, "No SAIL staked");

    // 按质押比例分配手续费
    address[] memory stakers = getStakers();
    for (uint i = 0; i < stakers.length; i++) {
        address staker = stakers[i];
        uint256 stakedAmount = stakingInfo[staker].amount;
        uint256 reward = feeAmount.mul(stakedAmount).div(totalStaked);

        _addPendingReward(staker, token, reward);
    }

    emit FeesDistributed(token, feeAmount);
}

/**
 * @dev 治理投票
 * SAIL持有者参与协议治理
 */

```

```

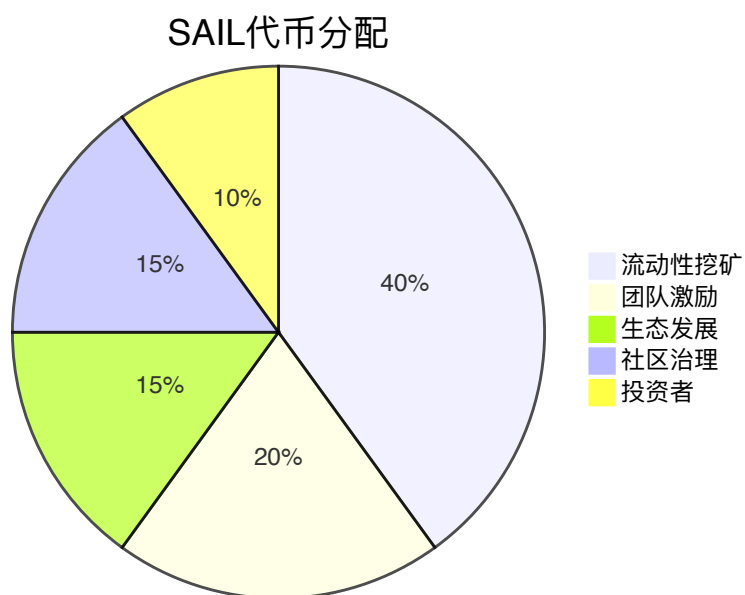
function vote(uint256 proposalId, bool support, uint256 votingPower) external {
    require(stakingInfo[msg.sender].amount >= votingPower, "Insufficient SAIL
balance");
    require(proposals[proposalId].active, "Proposal not active");

    if (support) {
        proposals[proposalId].forVotes =
proposals[proposalId].forVotes.add(votingPower);
    } else {
        proposals[proposalId].againstVotes =
proposals[proposalId].againstVotes.add(votingPower);
    }

    emit VoteCast(msg.sender, proposalId, support, votingPower);
}
}

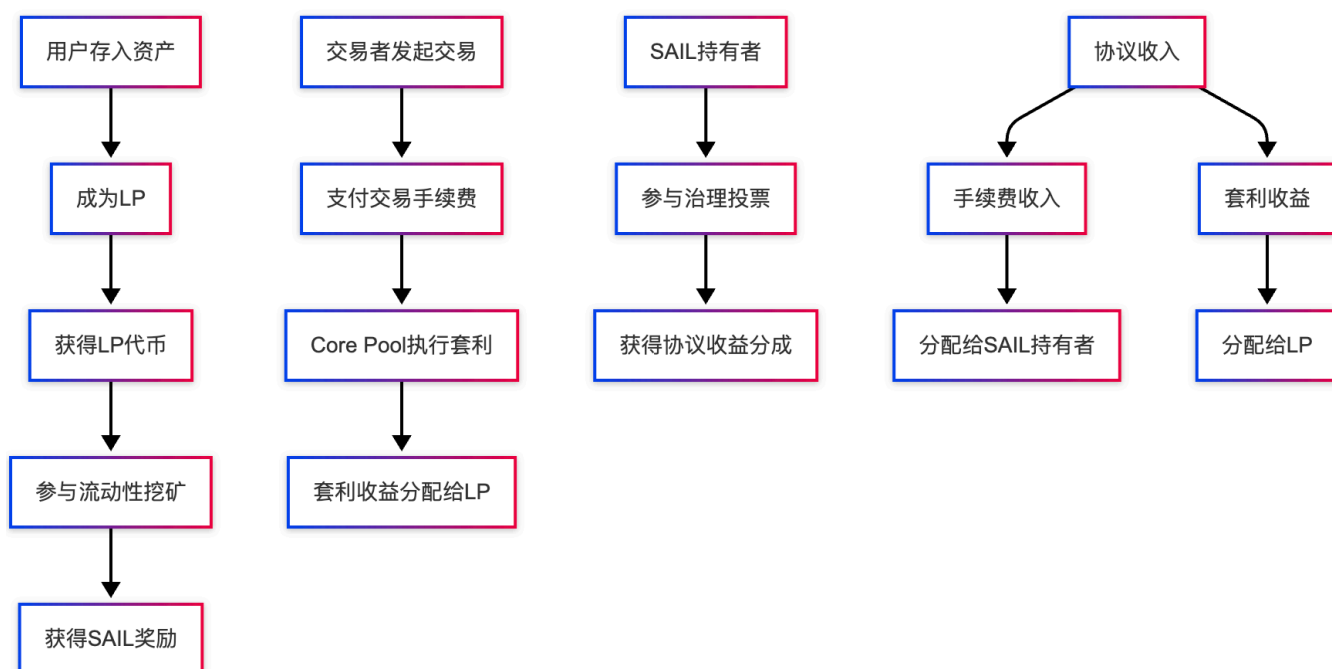
```

## 4.3 代币分配机制



## 5. 平台运营模式

### 5.1 业务流转流程



## 5.2 收益来源分析

### 5.2.1 交易手续费

- 费率：0.3%标准费率
- 分配：70%给LP，30%给SAIL持有者
- 月收入：基于\$2.3B总交易量，月均手续费约\$575万

### 5.2.2 套利收益

- 来源：Core Pool自动套利交易
- 频率：每日执行50-100次套利
- 收益率：年化15-25%额外收益

### 5.2.3 跨链桥接费

- 服务：支持7条链的资产桥接
- 费用：0.05-0.1%桥接费
- 分配：全部归协议金库

## 5.3 风险管理机制

```
// 风险管理合约
contract RiskManager {
    struct RiskParams {
        uint256 maxSlippage;           // 最大滑点
        uint256 maxPositionSize;       // 最大仓位
        uint256 liquidationThreshold;  // 清算阈值
        uint256 emergencyPause;        // 紧急暂停
    }
}
```

```
mapping(address => RiskParams) public tokenRiskParams;

/**
 * @dev 动态风险评估
 */
function assessRisk(address token, uint256 amount) external view returns (uint256 riskLevel) {
    uint256 volatility = getTokenVolatility(token);
    uint256 liquidity = getTokenLiquidity(token);
    uint256 positionRatio = amount.mul(100).div(getTotalLiquidity(token));

    // 综合风险评分
    riskLevel = volatility.add(positionRatio).sub(liquidity.div(10));

    if (riskLevel > 80) {
        return 3; // 高风险
    } else if (riskLevel > 50) {
        return 2; // 中风险
    } else {
        return 1; // 低风险
    }
}
```

## 6. 与主流DEX的对比分析

### 6.1 技术架构对比

| 特性    | Clipper   | Uniswap V3 | PancakeSwap | SushiSwap |
|-------|-----------|------------|-------------|-----------|
| 流动性模型 | 集中流动性+反套利 | 集中流动性      | 恒定乘积        | 恒定乘积      |
| 无常损失  | 通过套利消除    | 存在         | 存在          | 存在        |
| MEV保护 | 内置防护      | 部分保护       | 无           | 无         |
| 资金效率  | 25-100倍   | 4000倍      | 1倍          | 1倍        |
| 目标用户  | 小额交易者     | 全用户        | 全用户         | 全用户       |

### 6.2 核心优势分析

#### 6.2.1 相比Uniswap V3

优势：

- 无无常损失风险
- 自动化流动性管理
- 更好的MEV保护

劣势：

- 流动性深度较小
- 支持代币数量有限

### 6.2.2 相比PancakeSwap/SushiSwap

优势：

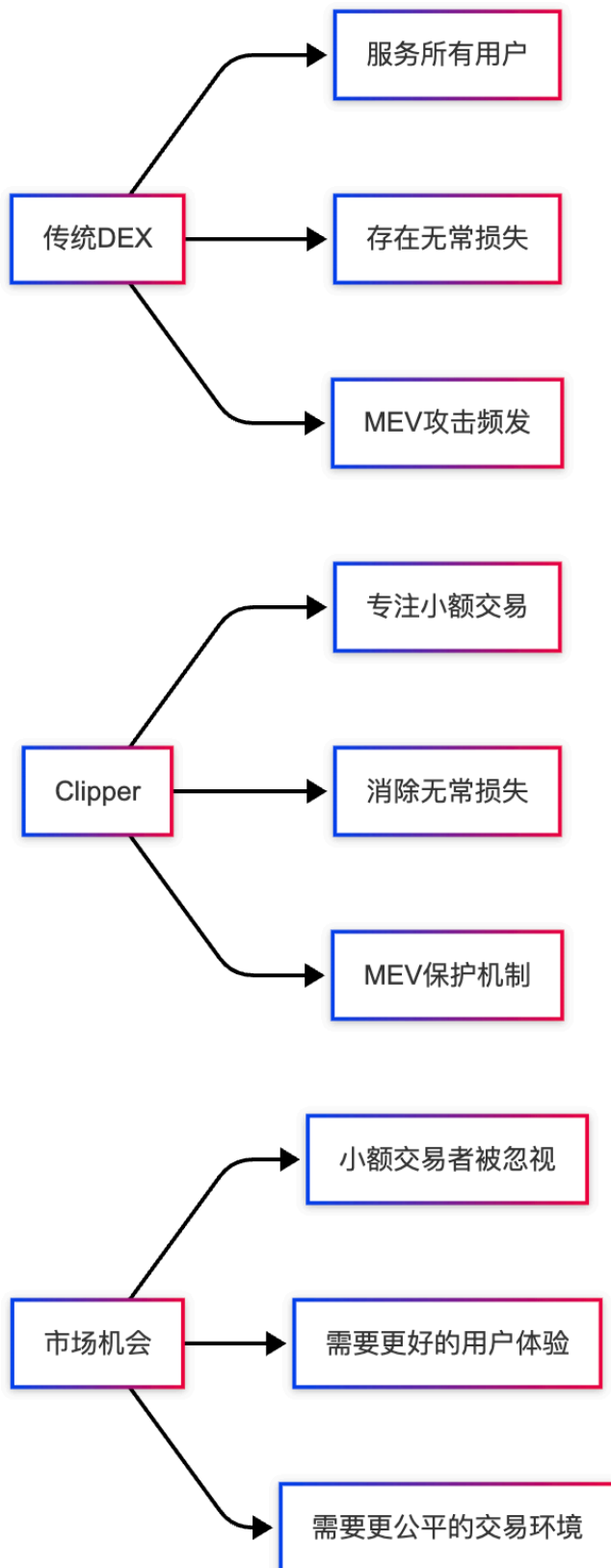
- 更高的资金利用效率
- 创新的反套利机制
- 专注小额交易优化

劣势：

- 生态规模较小
- 品牌知名度不足

## 6.3 市场定位差异





## 7. 技术实现细节

### 7.1 智能合约架构

```
// 主合约架构
contract ClipperExchange {
    // 核心组件
```

```

CorePool public corePool;
MEVProtector public mevProtector;
SAILTokenomics public sailTokenomics;
RiskManager public riskManager;

// 多链部署管理
mapping(uint256 => address) public chainDeployments;

/**
 * @dev 统一交易入口
 */
function swap(
    address tokenIn,
    address tokenOut,
    uint256 amountIn,
    uint256 minAmountOut,
    address to
) external returns (uint256 amountOut) {
    // MEV保护检查
    require(!mevProtector.detectMEVAttack(tokenIn, tokenOut, amountIn), "MEV attack
detected");

    // 风险评估
    uint256 riskLevel = riskManager.assessRisk(tokenIn, amountIn);
    require(riskLevel <= 2, "Risk level too high");

    // 执行交易
    amountOut = corePool.executeSwap(tokenIn, tokenOut, amountIn, minAmountOut);

    // 分配手续费
    _distributeFees(tokenIn, tokenOut, amountIn);

    emit Swap(msg.sender, tokenIn, tokenOut, amountIn, amountOut);
}
}

```

## 7.2 后端服务架构

```

// 套利引擎服务 (Golang)
type ArbitrageEngine struct {
    priceFeeds    []PriceFeed
    corePool      *CorePoolContract
    riskManager   *RiskManager
    logger        *Logger
}

func (ae *ArbitrageEngine) MonitorArbitrageOpportunities() {
    for {
        // 获取多个交易所价格
        prices := ae.fetchPricesFromMultipleExchanges()
    }
}

```

```

// 检测套利机会
opportunities := ae.detectArbitrageOpportunities(prices)

for _, opp := range opportunities {
    if ae.validateOpportunity(opp) {
        // 执行套利
        ae.executeArbitrage(opp)
    }
}

time.Sleep(time.Second * 5) // 5秒检测一次
}
}

func (ae *ArbitrageEngine) executeArbitrage(opp ArbitrageOpportunity) error {
    // 风险检查
    if !ae.riskManager.CheckRiskLimits(opp) {
        return errors.New("risk limits exceeded")
    }

    // 执行链上交易
    tx, err := ae.corePool.ExecuteArbitrage(opp.TokenA, opp.TokenB, opp.Amount)
    if err != nil {
        ae.logger.Error("Arbitrage execution failed", err)
        return err
    }

    ae.logger.Info("Arbitrage executed successfully", tx.Hash())
    return nil
}

```

## 7.3 前端集成示例

```

// React前端集成
import { ClipperSDK } from '@clipper/sdk';

const clipper = new ClipperSDK({
    chainId: 1, // Ethereum mainnet
    rpcUrl: 'https://mainnet.infura.io/v3/YOUR_KEY'
});

// 执行交易
async function executeSwap() {
    try {
        const quote = await clipper.getQuote({
            tokenIn: 'USDC',
            tokenOut: 'ETH',
            amountIn: '1000'
        });

        const tx = await clipper.swap({

```

```

        tokenIn: quote.tokenIn,
        tokenOut: quote.tokenOut,
        amountIn: quote.amountIn,
        minAmountOut: quote.minAmountOut,
        slippage: 0.5 // 0.5%
    });

    console.log('Transaction hash:', tx.hash);
} catch (error) {
    console.error('Swap failed:', error);
}
}

// 添加流动性
async function addLiquidity() {
    const result = await clipper.addLiquidity({
        tokenA: 'USDC',
        tokenB: 'ETH',
        amountA: '1000',
        amountB: '0.5'
    });

    console.log('LP tokens received:', result.lpTokens);
}

```